

PRODUCT BACKLOG

Project: Fábula Dental

Management System for a Pediatric Dental Clinic

Team: CodeXplorers | Version: 5.0 | Date: June 12th, 2026

Technologies (Unit 1 — Frontend): PHP · MongoDB · Bootstrap 5 · JavaScript · Docker

Technologies (Unit 2 — Backend APIs): Node.js · TypeScript · Express · Prisma · PostgreSQL (Supabase)

Team Members:

- Andrés Cárdenas
- Chuqui Kerly
- Díaz Victoria

1. Project Description

Fábula Dental is a web-based clinical management system specializing in pediatric dentistry. The platform allows users to manage patients, track payments, and manage dental supply inventory, with differentiated access based on user role (Administrator, Dentist, Receptionist).

In Unit 2, the project evolved from a monolithic PHP/MongoDB architecture to a distributed two-server TypeScript REST API system, maintaining the original frontend as a legacy reference while building a production-grade backend.

Technology Stack — Unit 1 (Legacy Frontend, archived in old-frontend/)

- Backend: PHP 8.x with MVC architecture
- Database: MongoDB Atlas (collections: patients, payments, supplies)
- UI: Bootstrap 5.3, Vanilla JavaScript, HTML5, CSS3
- Infrastructure: Docker (Dockerfile)
- Authentication: PHP sessions with role control

Technology Stack — Unit 2 (Current Backend Architecture)

- CRUD Server (port 3000) @ 34.58.145.37: Node.js + Express + TypeScript + Prisma ORM
- Business Logic Server (port 3001) @ 35.254.153.60: Node.js + Express + TypeScript + fetch API
- Database: PostgreSQL via Supabase (tables: patients, payments, supplies)
- ORM: Prisma with pg adapter (PrismaPg + pgbouncer connection pooling)
- Deployment: GCP — CRUD: 34.58.145.37 | Business Logic: 35.254.153.60

Architecture Overview

The system uses two decoupled servers:

- CRUD Server: Handles all direct database operations (CREATE, READ, UPDATE, DELETE) using Prisma to query PostgreSQL.
- Business Logic Server: Consumes the CRUD server via HTTP fetch calls, applies domain calculations (payment status, supply expiration, financial analysis, patient clinical logic) and returns enriched responses.

System Actors

- Administrator: Manages dental supply inventory and views financial analytics
- Dentist: Registers and manages patient information
- Receptionist: Registers and manages patient payments

1.5 Document Structure

This document is organized into three levels of requirements. Understanding their relationship is key to reading the backlog correctly:

Concept	Definition	Example in this document
Feature (F-XX)	High-level system capability that delivers value to a specific actor. Groups one or more related user stories into a coherent functional area.	<i>F-02 — Patient Management: covers all interactions a Dentist has with patient records.</i>
User Story (US-XX)	A specific requirement written from the actor's perspective following the format: "As [role], I want [action], for [benefit]." Defines what the actor needs and the acceptance criteria.	<i>US-02: 'As a Dentist, I want to register a new patient with ID, name, DOB, and reason for consultation, so that I have the complete medical history in the system.'</i>
Product Backlog Item (PB-XX)	A concrete, actionable development task derived from one or more user stories. Includes acceptance criteria, priority, story points, and current state (Done / Pending / Planned).	<i>PB-04: 'New patient registration' — implements US-02, High priority, 8 pts, Done. Specifies the POST /fabuladental/patients endpoint, validations, and HTTP responses.</i>

1.6 Features Overview

The table below groups all Product Backlog items into the eight high-level features of the system. Each feature is owned by one or more actors and cross-references the corresponding User Stories and PB items.

Feature	Description	Actor(s)	User Stories	PB Items
F-01	Authentication & Role-Based Access Control — Login, session management, and role-restricted module access.	All Actors	US-01	PB-01, PB-02, PB-03
F-02	Patient Management — Full CRUD of patient records with ID, age, legal representative, and consultation reason validations.	Dentist	US-02, US-03	PB-04, PB-05, PB-06, PB-07, PB-08, PB-17
F-03	Patient Business Logic — Server-side calculations on patient data: pediatric category classification, legal representative check, birthday countdown, consultation time, urgency level, and treatment complexity.	Dentist	US-02, US-03	PB-21, PB-22, PB-24, PB-26, PB-27, PB-31
F-04	Payment Management — Full CRUD of payment records with type (Deposit / Final), method (Cash / Card / Transfer), amount and date validations.	Receptionist	US-04, US-05, US-08	PB-09, PB-10, PB-11, PB-12

F-05	Payment Business Logic — Derived payment status, senior discount eligibility, contact priority score, outstanding patient balance, and daily revenue summary.	Receptionist	US-05, US-08	PB-23, PB-25, PB-28, PB-29
F-06	Inventory Management — Full CRUD of dental supplies with quantity, unit cost, order date, and expiration date validations and automatic status assignment.	Administrator	US-06, US-07	PB-13, PB-14, PB-15, PB-16
F-07	Inventory Analytics & Alerts — Financial analysis (asset valuation, restock budget, expiration losses, capital risk) and critical stock alerts.	Administrator	US-07, US-09	PB-15, PB-30
F-08	Security & Reporting — Encrypted credential storage, statistical dashboard, patient/inventory search, and PDF/Excel export.	All Actors	—	PB-17, PB-18, PB-19, PB-20

2. User Stories

User stories describe the functionalities of the system from the perspective of each actor, following the standard agile format: As [role], I want [action], for [benefit].

ID	As...	Wanna...	For...	Acceptance Criteria
US-01	Dentist /Receptionist/ Administrator	Log in with username and password	Access the module corresponding to my role	- Redirection based on role after successful login - Error message for invalid credentials - Fields cannot be empty
US-02	Dentist	Register a new patient with ID, name, Date of birthday, and reason for consultation, phone, gender.	Have the complete medical history in the system	- ID card with exactly 10 digits mandatory (HTTP 400 if violated) - legalRepresentative field required if patient < 18 years (HTTP 400 if missing) - Successful registration saves to PostgreSQL via CRUD API and returns 201
US-03	Dentist	View, edit, and delete patients from the list	Keep patient information up-to-date	- Table loads patients from GET /fabuladental/patients - Online editing sends PUT to the CRUD API - Deletion sends DELETE and removes row without reloading
US-04	Receptionist	Record a payment indicating patient, amount, type, and method, status,date.	Keep track of the clinic's income and payments	- Payment type: Deposit or Final - Final → Completed; Deposit → Partial - Amount > 0; date not future; Patient ID = 10 digits (HTTP 400 on any violation) - Method: Cash, Card, or Transfer
US-05	Receptionist	View the complete payment history with its current status	Have financial traceability	- All payments via GET /fabuladental/payments (BL Server) - Colors: Completed = green, Partial = yellow, Pending = red - Records can be edited and deleted
US-06	Administrator	Register dental supplies with id, name, quantity, unit cost, order Date, expiration Date, status.	Control the clinic's inventory	- Quantity min. 1 unit (HTTP 400 if < 1) - Expiration date >= order date (HTTP 400 if violated) - Missing required fields return HTTP 400 - Auto status: Current / NextExpiration / Expired
US-07	Administrator	View inventory with alerts for products nearing expiration	Anticipate replenishment needs	'NextExpiration' if expiration date <= 30 days from today 'Expired' if date has passed - Differentiated visual indicator in table
US-08	Receptionist	Retrieve payment history with calculated billing status	Provide financial traceability per patient	- GET /fabuladental/payments → derived status list (BL Server) - GET /fabuladental/payments/:paymentId → specific payment record - GET /fabuladental/payments/patients/:patientId → full patient history
US-09	Administrator	Perform financial analysis on inventory	Support administrative decision-making	- GET /fabuladental/supplies/asset-valuations → total inventory value (BL Server) - GET /fabuladental/supplies/restock-provisions → consolidated restock budget - GET /fabuladental/supplies/expiration-losses → total loss from expired items

ID	As...	Wanna...	For...	Acceptance Criteria
				GET /fabuladental/supplies/capital-risks → at-risk capital (NextExpiration items)

3. Product Backlog

The Product Backlog is ordered by epics and priority. Story points follow the Fibonacci scale (1, 2, 3, 5, 8).

High Priority	Average Priority	Low Priority	Done	Pending	Planned
---------------	------------------	--------------	------	---------	---------

3.1 Functional Backlog Items

ID	User Story	Description	Acceptance Criteria	Priority	Pts	State
PB-01	New patient registration	Dentist, I want to register a new patient with their personal details.	- Captures: ID (10 digits), full name, DOB, phone, reason for consultation - legalRepresentative field required if patient < 18 (HTTP 400 if missing for minor) - Stored via POST /fabuladental/patients (CRUD Server). Returns HTTP 201	High	8	Done
PB-02	List of patients	Dentist, I want to see all registered patients.	- Table via GET /fabuladental/patients - Individual record via GET /fabuladental/patients/:patientId (CRUD Server) - Each record can be edited or deleted	High	5	Done
PB-03	Patient data editing	Dentist, I want to edit an existing patient's data.	'Save' sends PUT /fabuladental/patients/:patientId (CRUD Server) - Changes reflected in real time	High	5	Done
PB-04	Patient removal	Dentist, I want to delete a patient record.	- DELETE /fabuladental/patients/:patientId (CRUD Server) - Record disappears without page reload	Average	3	Done
PB-05	Legal rep validation for minors	Dentist, I want the system to require a legal representative for patients under 18.	- Calculates age from DOB. Under 18 without guardian → HTTP 400 error - Exposed via POST /fabuladental/patients/legal-representative-validation (BL Server)	High	3	Done
PB-06	Payment record	Receptionist, I want to record a patient payment with type, method, and amount.	- Captures: Patient ID (10 digits), date, amount (> 0), type (Deposit/Final), method (Cash/Transfer/Card) - HTTP 400 if ID ≠ 10 digits, amount ≤ 0, or date is in the future - Status auto-calculated. Stored via POST /fabuladental/payments (CRUD Server)	High	8	Done
PB-07	Payment history	Receptionist, I want to see all recorded payments.	- Shows payments with status (Pending/Partial/Completed) via GET /fabuladental/payments (BL Server) - Records editable and deletable	High	5	Done
PB-08	Payment edition	Receptionist, I want to edit a recorded payment.	- Editable fields. Sends PUT /fabuladental/payments/:paymentId (CRUD Server) - Changes reflected immediately	Average	3	Done
PB-09	Visual indicator of payment status	Receptionist, I want to see payment status with distinctive colors.	- Pending = red/orange - Partial = yellow - Completed = green	Average	2	Pending

ID	User Story	Description	Acceptance Criteria	Priority	Pts	State
PB-10	Supply record	Administrator, I want to register a new dental supply.	- Captures: name, quantity, unit cost, order date, expiration date - Expiration date must be $\geq$ order date (HTTP 400 if violated) - HTTP 400 if quantity < 1 or required fields are missing - Stored via POST /fabuladental/supply (CRUD Server)	High	8	Done
PB-11	Inventory list with status	Administrator, I want to see the supply inventory with expiration status.	- Shows name, quantity, cost, dates - Status updated via POST /fabuladental/supplies/status-calculations (BL Server) - Statuses: Current, NextExpiration (≤ 30 days), Expired - GET /fabuladental/supplies/quantity-thresholds/:maxQuantity (CRUD) for low-stock filter - GET /fabuladental/supplies/statuses/:statusValue (CRUD) for status filter	High	5	Done
PB-12	Product expiration alert	Administrator, I want automatic identification of supplies about to expire.	Calculates expiration diff. If ≤ 30 days $\rightarrow$ NextExpiration with visual alert indicator	High	3	Done
PB-13	Editing and deleting supplies	Administrator, I want to edit or delete supplies.	- Edit: PUT /fabuladental/supplies/:id (CRUD Server) - Delete: DELETE /fabuladental/supplies/:id - List updates without page reload	Average	3	Done
PB-14	Patient search and filtering	Dentist, I want to search for patients by name or ID.	- Real-time search field filtering the table by name and ID number - Pending API route definition in URI spec	Average	5	Pending
PB-15	Dashboard with statistics	Administrator, I want summarized metrics on the dashboard.	- Cards with dynamic counters - Data from API in real time - Pending API route definition in URI spec	Low	8	Pending
PB-16	Secure database authentication	Administrator, I want encrypted user credentials in the database.	- Credentials stored in PostgreSQL with bcrypt hashing - Login queries DB instead of hardcoded values	High	8	Pending
PB-17	Exporting reports	Receptionist/admin, I want to export payment and inventory lists.	- Export button in payment and inventory lists - Generates downloadable PDF/Excel file - Pending API route definition in URI spec	Low	8	Planned
PB-18	BL — Birthday Countdown	Dentist, I want to know how many days are left until a patient's next birthday.	- POST /fabuladental/patients/days-to-birthday (BL Server) - Returns { daysUntilBirthday, isBirthdayWeek } - isBirthdayWeek = true if birthday is within 7 days - HTTP 400 if birthday format invalid	Low	2	Done
PB-19	BL — Senior Discount	Receptionist, I want to determine if a patient qualifies for a senior discount.	- POST /fabuladental/patients/senior-discount (BL Server) - Returns { isEligibleForDiscount, suggestedDiscountFactor } - Eligible if patient age ≥ 65 years	Average	2	Done

ID	User Story	Description	Acceptance Criteria	Priority	Pts	State
PB-20	BL — Consultation Time Estimate	Dentist, I want an estimated consultation time based on patient age and reason for consultation.	- HTTP 400 if birthday format invalid - POST /fabuladental/patients/consultation-time-estimation (BL Server) - Base time + extra minutes if age < 5, age > 70, or complex consultation reason - Returns { estimatedConsultationMinutes, baseMinutes } - HTTP 400 if birthday format invalid	Average	3	Done
PB-21	BL — Contact Priority	Receptionist, I want a contact priority score for a patient based on their contact details and urgency.	- POST /fabuladental/patients/contact-priority (BL Server) - Returns { contactPriorityScore, requiresImmediateAttention } - Score boosted by urgent keywords: dolor, urgencia, sangrado, emergencia - HTTP 500 on internal error	Average	3	Done
PB-24	BL — Pediatric Category	Dentist, I want the system to calculate the pediatric age category of a patient based on their date of birth, to support age-appropriate dosage and clinical decisions.	- POST /fabuladental/patients/pediatric-category (BL Server) - Returns { calculatedAgeYears, calculatedAgeMonths, pediatricCategory, dosageRestrictionFactor } - Maps patient to clinical age group (Infant, Toddler, Preschool, School-age, Adolescent) - HTTP 400 if birthday format is invalid - HTTP 500 on internal processing error	Average	2	Done

3.2 Technical Enablers

The following items were originally documented as User Stories PB-22 to PB-25 with "As the dev team..." framing. This is methodologically incorrect: User Stories must always be written from the perspective of a human business role receiving tangible value. Technical migration tasks are classified as Technical Enablers since the end user does not directly perceive a change in the graphical interface. They are relabeled TE-01 to TE-04.

ID	Technical Enabler	Description	Implementation Criteria	State
TE-01	TS Migration — Payments CRUD API	Migrate the payments module from PHP/MongoDB to TypeScript/PostgreSQL on the CRUD Server.	- paymentRoutes.ts in CRUD server (port 3000) - POST, PUT, DELETE, GET /fabuladental/payments endpoints - Validation: 10-digit patientId, amount > 0, non-future date, valid type and method - Status auto-calculated. Zero TypeScript errors (tsc --noEmit)	Done
TE-02	TS Migration — Payments Business Logic API	Expose payment business logic endpoints on the Business Logic Server.	- paymentsBusinessController.ts + paymentsBusinessRoutes.ts - GET /fabuladental/payments (history with derived status) - GET /fabuladental/payments/:paymentId (specific record) - GET /fabuladental/payments/patients/:patientId (patient history) - Consumes CRUD via fetch. Zero TS errors	Done

ID	Technical Enabler	Description	Implementation Criteria	State
TE-03	TS Migration — Supplies Business Logic API	Build financial analysis endpoints for inventory on the Business Logic Server.	- suppliesBusinessController.ts deployed and functional - POST /fabuladental/supplies/status-calculations - GET /fabuladental/supplies/asset-valuations - GET /fabuladental/supplies/restock-provisions - GET /fabuladental/supplies/expiration-losses - GET /fabuladental/supplies/capital-risks. Zero TS errors	Done
TE-04	TS Migration — Patients CRUD API	Migrate the patients module to TypeScript/Prisma on the CRUD Server.	- patientRoutes.ts in CRUD server (port 3000) - POST, GET, PUT, DELETE /fabuladental/patients endpoints - Validates 10-digit ID, legalRepresentative for minors, birthday format - Prisma ORM + BigInt IDs. Zero TS errors	Done

Total story points: 158 pts | Completed (Sprints 1–4): 127 pts | Pending/Planned (Sprints 5–6): 31 pts

4. Sprint Planning

The table below summarizes the distribution of Backlog items in development Sprints, with the objective of each iteration and the current progress status.

Sprint 4 updated to include new patient Business Logic items (PB-26–PB-31) and to reference Technical Enablers (TE-01–TE-04) instead of the old PB-22–PB-25 user story framing.

Sprint	Items Included	Sprint Objective	Pts	State
Sprint 1	PB-01, PB-02, PB-03, PB-04, PB-08, PB-17	Functional authentication, patient registration with validations, and public treatment page.	19	Done
Sprint 2	PB-05, PB-06, PB-07, PB-09, PB-10, PB-13, PB-14, PB-15	Complete CRUD for patients, payment registration, inventory management with expiration alerts.	37	Done
Sprint 3	PB-11, PB-12, PB-16	Payment and inventory refinement: editing, deletion, and visual status indicators. (Note: PB-12 still pending.)	8	Done
Sprint 4 (Unit 2)	TE-01, TE-02, TE-03, TE-04, PB-26, PB-27, PB-28, PB-29, PB-30, PB-31	Migration of all modules to TypeScript/PostgreSQL. Two-server architecture: CRUD (34.58.145.37:3000) and Business Logic (35.254.153.60:3001). Patient BL endpoints deployed on GCP.	55	Done
Sprint 5	PB-12, PB-18, PB-19, PB-20	Security: DB-stored credentials (bcrypt). Patient search. Statistical dashboard. Pending visual status indicator (PB-12).	23	Planned
Sprint 6	PB-20	Export of reports in PDF/Excel for payments and inventory.	8	Planned

5. Technical and Non-Functional Requirements

5.1 Implemented Functional Requirements

- RF-01: Complete CRUD of payments with types (Deposit/Final) and methods (Cash/Transfer/Card) — TypeScript + Prisma
- RF-02: Automatic derivation of payment status (Deposit → Partial, Final → Completed) in Business Logic layer
- RF-03: Complete CRUD of supplies with automatic expiration status calculation — TypeScript + Prisma
- RF-04: Automatic alerts for products with an expiration date ≤ 30 days (NextExpiration status)
- RF-05: Public treatment page accessible without authentication
- RF-06: RESTful APIs for each module (GET, POST, PUT, DELETE) — TypeScript/Express
- RF-07: Business Logic endpoints for financial analysis: asset valuation, restock provisioning, expiration losses, capital risk
- RF-08: Payment history endpoint with calculated status per payment record
- RF-09: Patient payment history endpoint filtered by patient ID
- RF-10: Legal representative validation endpoint for underage patients (PB-21)
- RF-11: Patient Business Logic endpoints — birthday countdown, senior discount, consultation time estimate, contact priority (PB-22–PB-25)
- RF-12: Patient urgency level assessment endpoint (BL Server) (PB-26)
- RF-13: Patient treatment complexity estimation endpoint (BL Server) (PB-27)
- RF-14: Outstanding patient balance summary endpoint per patient (BL Server) (PB-28)
- RF-15: Daily revenue summary endpoint grouped by payment method (BL Server) (PB-29)
- RF-16: Critical stock alert endpoint for inventory management (BL Server) (PB-30)
- RF-17: Pediatric category classification endpoint based on patient date of birth (BL Server) (PB-31)

5.2 Non-Functional Requirements

- RNF-01 Usability: Responsive interface with Bootstrap 5. Compatible with mobile and desktop.
- RNF-02 Performance: CRUD operations do not require full page reload (fetch API). BL communicates with CRUD via HTTP.
- RNF-03 Scalability: PostgreSQL (Supabase) with pgbouncer connection pooling for horizontal scaling.
- RNF-04 Portability: Deployable on GCP or any Node.js-compatible environment.
- RNF-05 Maintainability: Clean separation between CRUD and Business Logic servers. TypeScript strict mode enforced.
- RNF-06 Validation: Backend input validation — 10-digit IDs, positive amounts, non-future dates, valid enums, inventory business rules.
- RNF-07 Type Safety: All backend code compiled with TypeScript strict mode with zero type errors.

6. Definition of Done (DoD)

A backlog item is considered DONE when it meets all of the following criteria:

- The code is implemented and working in the development/production environment
- The acceptance criteria have been verified manually (Postman or browser)
- The functionality has been tested with real data in PostgreSQL (Supabase)
- There are no console errors or unhandled exceptions
- For TypeScript modules: compiles with zero errors using `tsc --noEmit`
- The code follows the project's architecture: `routes/` and `controllers/` under each server's `src/`
- The functionality has been reviewed and approved by the CodeXplorers team

7. API Endpoints Reference

Complete list of all URIs defined in the API Specification Document v1.3 (June 11, 2026 — CodeXplorers Team). The system uses two decoupled servers; each endpoint is tagged with its corresponding server type.

Legend:

[BL] = Business Logic Server (35.254.153.60:3001)

[CRUD] = CRUD Server (34.58.145.37:3000)

7.1 Business Logic Server — <http://35.254.153.60:3001>

Applies domain calculations and returns enriched responses. Consumes the CRUD server internally via HTTP fetch.

7.1.1 Supplies — Business Logic

Tag	Method	URI	Description
[BL]	POST	/fabuladental/supplies/status-calculations	Update supplies expiration status (Current / NextExpiration / Expired)
[BL]	GET	/fabuladental/supplies/asset-valuations	Get total inventory asset valuation (quantity × unitCost)
[BL]	GET	/fabuladental/supplies/restock-provisions	Get restock provisioning and consolidated budget (items ≤ 5 units)
[BL]	GET	/fabuladental/supplies/expiration-losses	Calculate total financial loss from expired supplies
[BL]	GET	/fabuladental/supplies/capital-risks	Get next-expiration capital at risk (NextExpiration items)

7.1.2 Patients — Business Logic

Tag	Method	URI	Description
[BL]	POST	/fabuladental/patients/pediatric-category	Calculate pediatric age category and dosage restriction factor
[BL]	POST	/fabuladental/patients/legal-representative-validation	Validate legal representative requirement for underage patients
[BL]	POST	/fabuladental/patients/days-to-birthday	Calculate days until next birthday and birthday week flag
[BL]	POST	/fabuladental/patients/senior-discount	Determine senior discount eligibility (age ≥ 65)
[BL]	POST	/fabuladental/patients/consultation-time-estimation	Estimate consultation duration based on age and reason complexity
[BL]	POST	/fabuladental/patients/contact-priority	Calculate contact priority score and immediate attention flag

7.1.3 Payments — Business Logic

Tag	Method	URI	Description
[BL]	GET	/fabuladental/payments	Get payment history with calculated status (Pending / Partial / Completed)
[BL]	GET	/fabuladental/payments/:paymentId	Retrieve complete details of a specific payment by ID
[BL]	GET	/fabuladental/payments/patients/:patientId	Retrieve all payment records for a specific patient by DNI

7.2 CRUD Server — <http://34.58.145.37:3000>

Handles all direct database operations via Prisma ORM against PostgreSQL (Supabase).

7.2.1 Supplies — CRUD

Tag	Method	URI	Description
[CRUD]	GET	/fabuladental/supplies	Get complete list of all dental supplies
[CRUD]	GET	/fabuladental/supplies/quantity-thresholds/:maxQuantity	Get supplies filtered by maximum stock quantity
[CRUD]	GET	/fabuladental/supplies/statuses/:statusValue	Get supplies filtered by status (Expired / NextExpiration / Current)
[CRUD]	POST	/fabuladental/supply	Register a new dental supply into inventory
[CRUD]	PUT	/fabuladental/supplies/:id	Update an existing supply's attributes by ID
[CRUD]	DELETE	/fabuladental/supplies/:id	Remove a dental supply from inventory by ID

7.2.2 Payments — CRUD

Tag	Method	URI	Description
[CRUD]	POST	/fabuladental/payments	Record a new patient payment (Deposit or Final)
[CRUD]	PUT	/fabuladental/payments/:paymentId	Update a recorded payment to correct errors or status
[CRUD]	DELETE	/fabuladental/payments	Delete a payment record by ID (sent in request body)

7.2.3 Patients — CRUD

Tag	Method	URI	Description
[CRUD]	POST	/fabuladental/patients	Register a new patient with ID, name, birthday, and consultation reason
[CRUD]	GET	/fabuladental/patients	Retrieve full list of all registered patients
[CRUD]	GET	/fabuladental/patients/:patientID	Retrieve full record of a single patient by DNI
[CRUD]	PUT	/fabuladental/patients/:patientId	Update an existing patient's information by DNI
[CRUD]	DELETE	/fabuladental/patients/:patientId	Delete a patient record by DNI

Total endpoints: 28 | Business Logic [BL]: 14 endpoints (5 Supplies + 6 Patients + 3 Payments) | CRUD [CRUD]: 14 endpoints (6 Supplies + 3 Payments + 5 Patients)